Project Report Finding Similar Items

Francesco Memmola

Academic Year 2025/2026

Master in Data Science for Economics
Università degli Studi di Milano

Project Repository: `https://github.com/FraSeb6/Similarity_Ish_Abstract`

1 Introduction

The goal of this project is to detect pairs of highly similar abstracts inside the arXiv corpus (roughly 3 million scientific papers). The difficulty is not conceptual but computational: the similarity measure of interest is the **Jaccard similarity** between abstracts, and comparing every possible pair would cost $O(n^2)$ — about 4.5×10^{12} comparisons on the full corpus, which is computationally infeasible.

The classical **Shingling** $\rightarrow$ **MinHash** $\rightarrow$ **LSH** pipeline is therefore adopted, estimating Jaccard similarity through compact signatures and restricting comparisons to document pairs that are already likely to be similar. Two design priorities guided every choice:

- Each step is implemented to reflect its theoretical definition as closely as possible, making the connection between the algorithm and its mathematical foundation explicit and verifiable.
- Both **precision** and **recall** are measured against an exact ground truth, and the resulting pairs are grouped into **duplicate clusters**. As shown in Section 3, measuring recall is precisely what allowed the diagnosis and correction of a poor parameter choice.

A single global flag, `USE_SUBSAMPLE`, switches the same code between a fast subsample (used for the numbers reported here) and the complete dataset.

2 Data Ingestion and Cleaning

The arXiv metadata snapshot (`Cornell-University/arxiv`) is downloaded through the Kaggle API as a single $\sim$4 GB file with one JSON object per line.

Streaming. The file is never loaded into memory as a whole. We iterate over it *line by line*, parse each record, keep only the `abstract` field, and append it to a growing list. Memory therefore depends only on how many abstracts we decide to keep, not on the file size.

Quality filter. Very short abstracts — placeholders and "this paper has been withdrawn" notices — produce tiny shingle sets that generate spurious high-similarity matches. We drop any abstract shorter than 20 words directly inside the streaming loop. On the 50,000-line working slice this removes 673 entries, leaving **49,327** clean abstracts. The kept records are collected into a `pandas` DataFrame.

Normalisation. Before shingling, each abstract is lower-cased and all runs of whitespace (spaces, tabs, newlines) are collapsed to a single space. We deliberately keep normalisation light: it is enough to remove formatting noise while staying simple to justify.

3 From Abstracts to Sets: Character Shingling

Each abstract is turned into the **set of its k-character shingles** — every substring of k consecutive characters. Two abstracts that share many shingles are similar, so the Jaccard similarity of these sets is exactly the quantity we want.

3.1 Choosing k by experiment

The value of k is the single most consequential parameter, and we fixed it empirically rather than by assumption.

With $k = 5$ the shingles are too generic: fragments such as `" the"`, `"tion"` or `"ing "` occur in almost every English abstract. Their minimum-hash values then coincide across unrelated documents, and LSH was flooded with false positives — roughly 50,000 candidate pairs, a precision of only a few percent, and one connected "cluster" of 681 entirely unrelated papers.

With $k = 9$ a shingle is rare enough to be meaningful. Unrelated abstracts stop colliding: the candidate set shrank to a few hundred pairs, precision rose to about 63%, recall stayed high, and the largest cluster became a genuine family of near-identical papers. We adopt $k = 9$.

3.2 Hashing the shingles

Storing millions of string shingles would be wasteful, so each shingle is hashed to an integer with Python's `zlib.crc32`, keeping the lower 31 bits (a bitwise AND with `0x7FFFFFFF`). This guarantees every shingle value stays below the prime used by MinHash, which keeps the later modular arithmetic exact. The unique integers are stored in a Python `set`, so each shingle is counted once.

4 MinHash Signatures

Comparing full shingle sets pairwise is infeasible, so MinHash compresses each set into a fixed-length **signature** of $M = 100$ integers.

4.1 Principle

For a random permutation h of the shingle universe, the probability that two sets share the same minimum element equals their Jaccard similarity:

$$P\big(\min h(A) = \min h(B)\big) = J(A, B). \tag{1}$$

Using M independent permutations, the fraction of matching signature positions estimates J. Permutations are simulated by a linear hash family

$$h_{a,b}(x) = (a \cdot x + b) \bmod p, \tag{2}$$

where x is a 31-bit shingle integer, $p = 2^{31} - 1$ is a (Mersenne) prime, and a, b are random coefficients drawn once with a fixed seed for reproducibility. Keeping both the coefficients and the shingles below 2^{31} ensures that the product $a \cdot x$ never overflows 64-bit integers, so no approximation is introduced.

4.2 Implementation

We favour a transparent implementation over an aggressively vectorised one. The signature of a document starts with every slot at $+\infty$; for each shingle x we evaluate all 100 hash functions at once (a single NumPy expression over the coefficient arrays) and keep the element-wise minimum:

```
sig = np.minimum(sig, (A * x + B) % PRIME)
```

This is the literal definition of MinHash — a minimum over the document's shingles — and is trivial to defend, while NumPy still vectorises the work across the 100 functions. Signatures are stored in an in-memory integer matrix of shape $N \times 100$.

5 Locality-Sensitive Hashing with a Derived Split

Even with short signatures, comparing all of them is still $O(n^2)$. LSH avoids this by hashing similar signatures into shared buckets.

5.1 Banding

The $N \times 100$ signature matrix is split into b bands of r rows each, with $b \cdot r = 100$. Two documents that are identical in at least one band fall into the same bucket and become a **candidate pair**. For a pair of true similarity s, the probability of becoming a candidate follows the S-curve

$$P(\text{candidate}) = 1 - (1 - s^r)^b, \tag{3}$$

whose steep point sits near the threshold

$$t \approx \left(\tfrac{1}{b}\right)^{1/r}. \tag{4}$$

5.2 Deriving b and r

Rather than hard-coding the banding, we *derive* it from a target threshold. Given `TARGET_THRESHOLD` $= 0.5$, the code enumerates every factorisation of 100 into $b \cdot r$ and keeps the one whose threshold is closest to the target. This selects $b = 20$ **bands of** $r = 5$ **rows**, with

$$t = \left(\tfrac{1}{20}\right)^{1/5} \approx 0.549. \tag{5}$$

The resulting S-curve is shown in Figure 1.

5.3 Bucketing

Processing one band at a time, we take each document's 5-integer slice, convert it to a raw byte string with `tobytes()`, and use it as a dictionary key grouping the documents that share it. For every bucket with at least two documents, `itertools.combinations` yields the candidate pairs. On the working slice this produces **380 candidate pairs** in about one second.

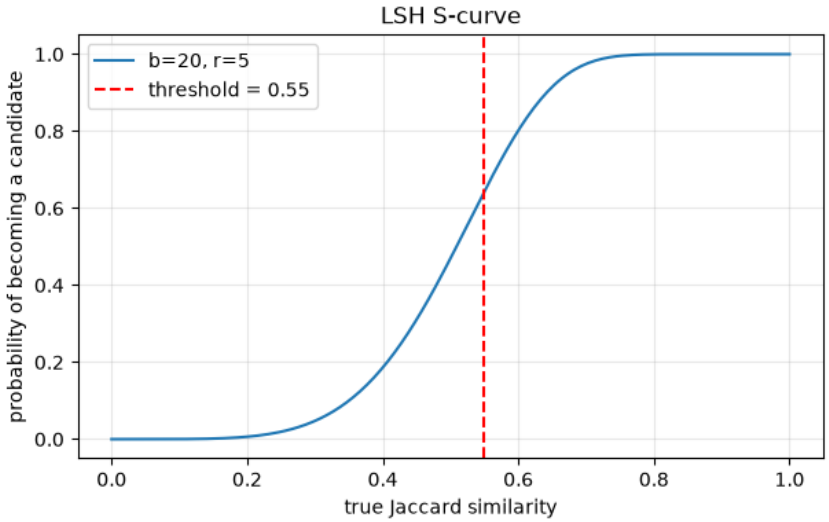

Figure 1: LSH S-curve for the derived split ($b = 20$, $r = 5$). The dashed line marks the threshold $t \approx 0.549$: pairs above it are almost always retained, pairs well below it are almost always discarded.

6 Evaluation: Precision and Recall

LSH is approximate, so its output must be validated. Crucially, we measure not only how many returned pairs are correct (**precision**) but also how many true pairs were found (**recall**) — precision alone cannot reveal the pairs that were missed.

6.1 Exact ground truth

On a small subsample of 4,000 documents we compute the *exact* Jaccard similarity of *all* pairs by brute force. This is $O(n^2)$ and only affordable on a small slice, but it gives the true set of similar pairs: at threshold 0.549 there are **7** of them.

6.2 Results

Restricting the candidates to the same 4,000-document slice, LSH recovers 6 of the 7 true pairs, i.e. a **recall of** 85.7%. Because this slice contains only a handful of candidates, we also compute a more robust **global precision** over *all* 380 candidate pairs, recomputing their exact Jaccard: 63.2% of them are genuinely similar. Figure 2 shows the distribution of exact similarities for the full candidate set, with a clear mass of true duplicates above the threshold and a tail of false positives below it.

7 Duplicate Clusters

Candidate pairs describe a graph in which each abstract is a node and each pair is an edge. The **connected components** of this graph are groups of mutually near-duplicate abstracts, which we extract with a simple depth-first traversal (no external library). The working slice yields **337** such groups. The largest contains 7 abstracts (arXiv IDs `0710.4022`–`0710.4028`): a literal "series of seven papers" sharing almost identical text — a convincing qualitative confirmation that the pipeline isolates real duplicates rather than statistical noise.

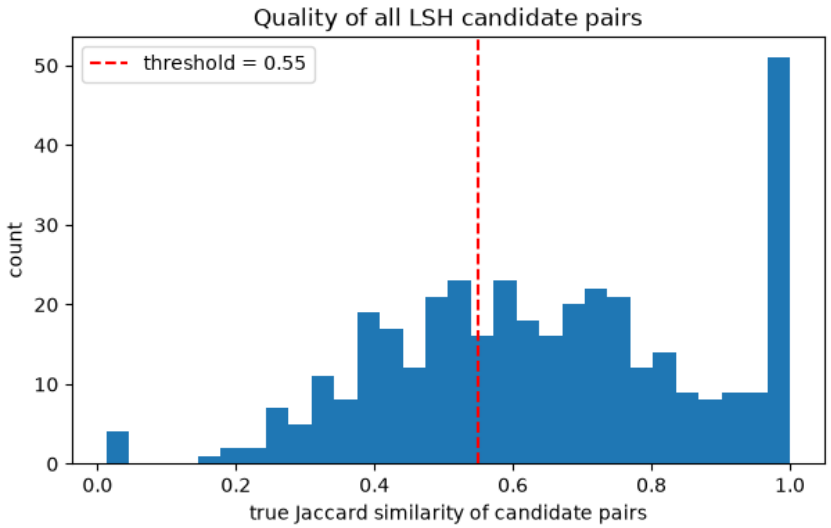

Figure 2: Exact Jaccard similarity of all 380 candidate pairs. The dashed line is the LSH threshold; candidates to its right are true near-duplicates.

8 Scalability

To show the pipeline scales, we time MinHash and LSH on growing prefixes of the data and append, as a final point, the run on the whole loaded dataset — reusing the times already measured in the main pipeline, so this last row costs nothing extra. Results are in Table 1 and Figure 3.

Documents (n)	**MinHash**	**LSH**	**Total**	**Candidates**
5,000	5.3 s	0.0 s	5.4 s	15
10,000	10.9 s	0.2 s	11.1 s	35
20,000	21.6 s	0.4 s	22.0 s	88
40,000	42.8 s	0.7 s	43.5 s	298
49,327	51.2 s	1.1 s	52.3 s	380

Table 1: Execution time per stage on increasing dataset sizes. Time grows linearly with n.

Both stages grow **linearly** in n: total time roughly doubles when n doubles, confirming we have bypassed the $O(n^2)$ barrier. MinHash dominates; LSH is negligible. Extrapolating the slope, the full ∼3M-abstract corpus is processed in on the order of 50 minutes for MinHash, by simply setting `USE_SUBSAMPLE = False` — the same code, unchanged.

Limitations. The metrics above are measured on a 50,000-abstract subsample; recall in particular is estimated from a small ground truth, so it carries sampling noise. The readable MinHash is intentionally not the fastest possible implementation. Finally, recall below 100% is the expected price of LSH: a small fraction of true pairs can be missed when they never collide in any band.

9 Conclusion

We implemented a complete near-duplicate detection pipeline for arXiv abstracts using Shingling, MinHash, and LSH over the Jaccard similarity. Writing each stage close to its definition kept the code fully explainable, while a dual precision/recall evaluation and a clustering step

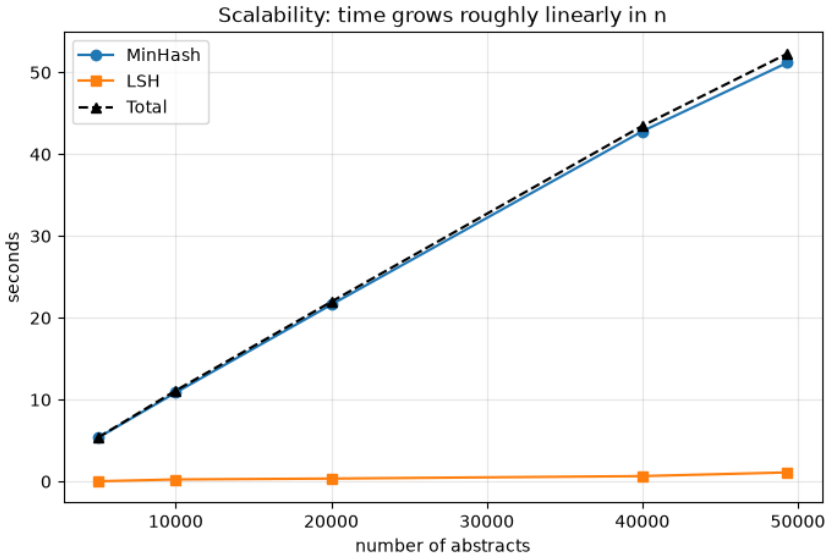

Figure 3: MinHash and LSH execution time versus the number of abstracts.

turned the raw candidate pairs into interpretable, validated results. The empirical choice of 9-character shingles (63.2% global precision, 85.7% recall) and the data-driven 20×5 banding illustrate that careful parameter selection, guided by measurement, matters as much as the algorithm itself. The pipeline scales linearly and runs on the full dataset through a single switch.

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work, and including any code produced using generative AI systems. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.